TASK 01 — API Architecture Review & Contract Design

Step 1: Review existing User, Order, Product, and Payment services

understand the purpose of each microservice.

User Service

Manages user registration, profiles, and user information.

Example: Get user details using a user ID.

Product Service

Manages product names, prices, descriptions, and availability.

Example: Retrieve the price of a mobile phone.

Order Service

Manages order creation, order details, and order status.

Example: Create an order for two mobile phones.

Payment Service

Manages payment requests, transaction results, and payment status.

Example: Process a payment for an order.

What should you do

1. Open the existing Spring Boot project.
2. Find the controller classes.
3. Find the service and repository classes.
4. Check application.properties or application.yml.
5. Identify each service's port, endpoints, and database.
6. Run the services and check whether they work.

Output: A document describing the four services and their responsibilities.

Step 2: Identify service boundaries and dependencies

Identify what each service owns and how it communicates with other services.

Example, the Order Service may call the Product Service to verify a product's price and the Payment Service to initiate payment.

Order Service

Creates orders

| Checks product details

Product Service

|Payment request

Payment Service

What should you check

- Does Order Service call Product Service?
- Does Order Service call Payment Service?
- Are services communicating through REST APIs or messaging systems such as Kafka?
- Does each service own its database?
- What happens if a dependent service is unavailable?

Output: A dependency table and a diagram showing service communication.

Step 3: Document API endpoints, methods, request bodies, and responses

Record the details of every existing API so developers know how to use it.

An API contract defines how a client sends a request and what response it can expect.

Example: Create an order

Request:

POST /api/v1/orders

Content-Type: application/json

Request body:

```
{
  "userId": 10,
  "productId": 101,
```

```
  "quantity": 2
}
```


Example successful response:


```
{
  "orderId": 5001,
  "userId": 10,
  "productId": 101,
  "quantity": 2,
  "status": "PENDING"
}
```


HTTP methods

Method	Purpose	Example
GET	Retrieve data	Get product details
POST	Create or initiate an operation	Create an order
PUT	Replace a resource	Update a product
PATCH	Partially update a resource	Update order status
DELETE	Delete a resource	Delete a product

What should you document

For every API, write:

1. Endpoint URL.
2. HTTP method.
3. Request parameters and body.
4. Response body and status code.
5. Validation rules.
6. Error responses and authentication requirements.

Output: API documentation, preferably in Swagger/OpenAPI format.

Note: The endpoint and JSON above are illustrative examples. Use the actual endpoints and fields from your project.

Step 4: Define standard API error responses

Make errors consistent and easy to understand across all microservices.

Example, when a product does not exist, the API should return a suitable error instead of an unclear message.

Example:


```
{
  "status": 404,
  "error": "Not Found",
  "message": "Product not found",
  "path": "/api/v1/products/999"
}
```


Common HTTP error codes

Code	Meaning	Example
400	Bad Request	Invalid quantity
401	Unauthorized	Invalid authentication
403	Forbidden	User lacks permission
404	Not Found	Product does not exist
409	Conflict	Conflicting order state
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unexpected application error
503	Service Unavailable	Service is temporarily unavailable
504	Gateway Timeout	Upstream service took too long

In Spring Boot

Use @RestControllerAdvice and @ExceptionHandler to handle exceptions and return a standard error format.

Output: A standard error-response structure for all services.

Step 5: Identify timeout and service failure scenarios

Find out what happens when a microservice is slow, unavailable, or unable to complete a request.

Example

1. A customer places an order.
2. Order Service sends a request to Payment Service.
3. Payment Service does not respond within the configured timeout.
4. Order Service handles the timeout.
5. The client receives an appropriate response, or the order remains pending while payment status is checked.

failure scenarios

Failure	Meaning	Recommended handling
Connection timeout	Connection cannot be established in time	Configure connection timeouts
Read timeout	Response takes too long	Configure read timeouts
Service unavailable	Target service cannot handle requests	Use a circuit breaker and appropriate error handling
Duplicate payment request	The same operation is submitted again	Use idempotency protection
Network failure	Services cannot communicate	Retry suitable operations carefully
Partial failure	One operation succeeds while another fails	Use recovery and reconciliation logic

A payment timeout does not necessarily mean the payment failed. Check its status before attempting another charge.

Output: A report explaining each failure, its impact, and the recommended recovery strategy.

Step 6: Draw architecture and request-flow diagrams

Create diagrams that explain the system's structure and how requests travel through it.

A. Architecture diagram

Shows the main components and their connections.

Client / Frontend

API Gateway

User Service

Product Service

Order Service

Payment Service

Illustrative architecture. Add actual dependencies and databases from your application.

B. Request-flow diagram

Shows the sequence of actions when a customer places an order:

1. Client sends the order request.
2. API Gateway routes the request.
3. Order Service validates the request.
4. Product Service supplies product information.
5. Payment Service processes the payment.
6. Order Service records the appropriate order status.
7. The client receives the response.

You can draw these diagrams using draw.io or represent them using Mermaid.

Output: Architecture diagram and request-flow diagram.

Step 7: Test existing APIs using Postman

Verify that the APIs return the expected responses.

Step-by-step process

1. Open Postman.
2. Create a collection named Microservices API Review.
3. Create folders for User, Product, Order, and Payment.
4. Enter the actual API URL and select the HTTP method.
5. Add authentication, headers, or a request body if required.
6. Click Send.
7. Check the HTTP status code, response body, and response time.
8. Record whether the test passed or failed.

Example request:

GET http://localhost:8083/api/v1/products/101

This example assumes Product Service runs on port 8083 and exposes that endpoint. Verify the actual configuration before testing.

If product 101 exists, the expected result might be 200 OK. If it does not exist, the API might return 404 Not Found.

Test both successful and unsuccessful scenarios, including invalid input, missing authentication, nonexistent resources, and service timeouts.

Output: Postman collection, test results, and a list of confirmed defects.

Step 8: Document findings and improvement areas

Prepare a final report explaining what you discovered and how the system can be improved.

Example:

Finding to investigate	Possible improvement
API errors differ between services	Standardize error responses
Timeouts are not configured appropriately	Configure connection and read timeouts

Duplicate requests can create duplicate payments	Introduce idempotency controls
API documentation is incomplete	Maintain OpenAPI specifications
Requests are difficult to trace	Use correlation IDs and distributed tracing

These are examples of possible issues, not confirmed problems in your application. Include only verified findings in your actual report.

Output: A final review report with findings, impact, evidence, and recommendations.

Final checklist

Reviewed User, Order, Product, and Payment services

Identified service boundaries and dependencies

Documented existing API contracts

Defined standard error responses

Reviewed timeouts and service failures

Created architecture and request-flow diagrams

Tested APIs using Postman

Prepared findings and recommendations